

PYTHON DAY 7

Dictionaries, Tuples, Sets & Choosing the Right Collection

Important: Day 7 is intentionally a **concept-first day**. There is no code in this guide. Understand, compare, draw, and design the data structures before writing Python.

Goal: Understand when to use a list, tuple, set, or dictionary and how these structures represent real-world data.

1. Warm-up: Think Like a Data Designer

Imagine a banking application. You need a customer's name, account number, balance, transaction history, and permissions. Ask: should everything be one list? Should some values be identified by names rather than positions? Should duplicates be allowed? Which information needs ordering?

These questions are why Python provides multiple collection types.

2. The Four Core Collection Types

List: ordered and mutable. Think: a sequence of items.

Tuple: ordered and immutable. Think: a fixed group of values.

Set: unique values. Think: membership and uniqueness.

Dictionary: key-value mapping. Think: labeled data and lookup.

3. Dictionaries: Data With Meaning

A dictionary stores information using keys and values. Instead of thinking 'the second item means email', you give the field a meaningful name such as email.

Mental model: a dictionary is similar to a form, record, or lookup table.

Keys identify values and are unique within a dictionary. Values can be almost any Python object, including lists, tuples, sets, and other dictionaries.

You should understand conceptually how to create a record, retrieve a value by key, add a field, update a field, remove a field, test whether a key exists, and inspect keys, values, and key-value pairs.

4. Why Dictionaries Matter

Lists answer questions like 'what is item number 3?' Dictionaries answer questions like 'what is this user's email?'

This distinction becomes important in APIs, configuration, databases, authentication systems, analytics, and almost every non-trivial application.

5. Dictionary Lookup and Data Modeling

Imagine 100,000 users. If your main operation is finding a user by user ID, a key-based mapping is conceptually more appropriate than treating the whole collection as a positional sequence.

Do not memorize performance numbers today. Learn the engineering principle: **choose a data structure based on the operations your application needs.**

6. Nested Dictionaries

Real records are often hierarchical. A user can have profile information, preferences, permissions, and an address. Dictionaries can contain other collections as values.

This is also the conceptual foundation for understanding JSON objects returned by many web APIs.

7. Tuples: Ordered but Fixed

A tuple is an ordered collection that cannot be modified after creation.

Mental model: a fixed record or package of related values.

Tuples make sense when the number and meaning of positions are stable, such as coordinates or a fixed group of related values.

The key idea is **immutability**: the tuple itself cannot have its elements reassigned after creation.

8. List vs Tuple

Both: ordered and allow duplicates.

List: mutable, suitable for changing collections.

Tuple: immutable, suitable for fixed collections or fixed records.

Ask yourself: 'Will this collection change?' If yes, a list may be appropriate. If the group should remain fixed, a tuple may communicate that intent better.

9. Sets: Uniqueness First

A set represents a collection of unique elements. When uniqueness matters more than position, a set is often the natural model.

Sets are useful for membership checks and mathematical operations such as union, intersection, difference, and symmetric difference.

Do not design a set around positions. A set is about membership, not 'item number 3'.

10. Real-World Set Example

Imagine Team A has permissions A, B, and C while Team B has B, C, and D.

The intersection represents shared permissions. The union represents all permissions present. The difference represents permissions belonging to one team but not the other.

This is much clearer conceptually than manually comparing two lists.

11. Choosing the Right Collection

Need ordered data that changes? Think list.

Need a fixed ordered group? Think tuple.

Need uniqueness and set operations? Think set.

Need labeled fields or key-based lookup? Think dictionary.

The choice should come from the data and the operations, not from which structure you learned most recently.

12. Design Exercise: Model a User

Design a user without writing code. Required information: user ID, name, email, roles, skills, login history, and account status.

Decide which information should be represented by a dictionary, list, tuple, or set. Explain every decision.

Research question: can one user record contain multiple collection types at the same time? Think about why real data models are usually combinations of structures.

13. Design Exercise: E-Commerce Product

Model an online product conceptually: product ID, name, price, categories, available sizes, specifications, and customer reviews.

Choose a collection for each field. Pay attention to whether order, uniqueness, or meaningful labels matter.

14. Design Exercise: Social Media Post

Design a social post containing post ID, author, text, hashtags, likes, comments, and timestamp.

Identify which values are singular and which are collections. Decide where uniqueness matters and where labeled fields matter.

15. Practical Project: Contact Book

Design a CLI contact book using dictionaries.

Requirements: add a contact, view a contact, update contact information, delete a contact, list contacts, and search.

Important: Before coding, draw the data structure on paper. Decide what the outer structure is and what each contact contains.

Upgrade: Add multiple phone numbers, contact groups, and favorite contacts. Decide where lists and sets make sense.

16. Main Project: Student Management System

Design a student management system before writing code.

Student information: ID, name, email, subjects, marks, attendance, skills, and active status.

Operations: add, find, update, remove, list, calculate performance, find unique skills, and identify students enrolled in a subject.

Architecture challenge: use the smallest number of collection types that still make the data easy to understand and operate on.

17. Research Lab

Research and explain these in your own words:

1. Why must dictionary keys be hashable?
2. Why are lists mutable while tuples are immutable?
3. Why are sets useful for uniqueness?
4. What does 'hashable' mean?
5. Why can a tuple contain a list even though the tuple itself is immutable?
6. Why can dictionaries and sets be efficient for lookup and membership?
7. What is the difference between a collection and a mapping?
8. How are JSON objects conceptually related to Python dictionaries?
9. When would a list be a poor design choice?

10. When would a dictionary be unnecessarily complicated?

18. No-Code Debugging Lab

Do not write code. Choose the best data structure and explain why.

- A. Store 50 email addresses and guarantee no duplicates.
- B. Store latitude and longitude as a fixed pair.
- C. Look up a customer's address using customer ID.
- D. Maintain a changing ordered queue of tasks.
- E. Compare permissions between two teams.
- F. Represent a product with name, price, category, and stock quantity.

19. The Engineering Habit

Before choosing a collection, ask: Does order matter? Will the data change? Do duplicates matter? Do I need lookup by a meaningful key? Do I need membership checks? Is this a fixed group of related values? Will another developer understand my choice?

Good software design often starts with choosing a good representation of the data.

20. Knowledge Check

You should be able to explain, without code:

- List vs tuple
- List vs set
- Set vs dictionary
- Keys and values
- Mutability vs immutability
- Uniqueness
- Membership
- Key-based lookup
- Nested collections
- How to justify a collection choice

21. Completion Checklist

- I can explain the purpose of a dictionary.
- I understand keys and values conceptually.
- I understand dictionary lookup.
- I understand nested collections.
- I understand tuple immutability.
- I understand why sets focus on uniqueness.
- I can compare list, tuple, set, and dictionary.
- I completed all no-code design exercises.
- I designed the Contact Book data model.
- I designed the Student Management System data model.

■ I completed the research lab.

■ I can justify a data-structure choice.

22. Recommended Day 7 Schedule

30 min: Learn the mental models.

40 min: Compare the four collection types using real-world examples.

45 min: Complete the design exercises.

60 min: Design the Contact Book on paper.

60–90 min: Design the Student Management System data model.

30 min: Research lab and no-code debugging.

Rule for today: Do not rush into syntax. If you can explain the data model clearly on paper, writing the Python later becomes much easier.

23. Official References to Read

Python Documentation: Built-in Types

Python Documentation: Mapping Types

Python Documentation: Set Types

Python Documentation: Tuples and Sequences

Use the official Python documentation as the primary reference. The goal is to learn how to read documentation, not just follow tutorials.

24. Day 8 Preview

Functions Deep Dive: function parameters, return values, scope, local vs global variables, default arguments, keyword arguments, and designing small functions that do one clear job.